

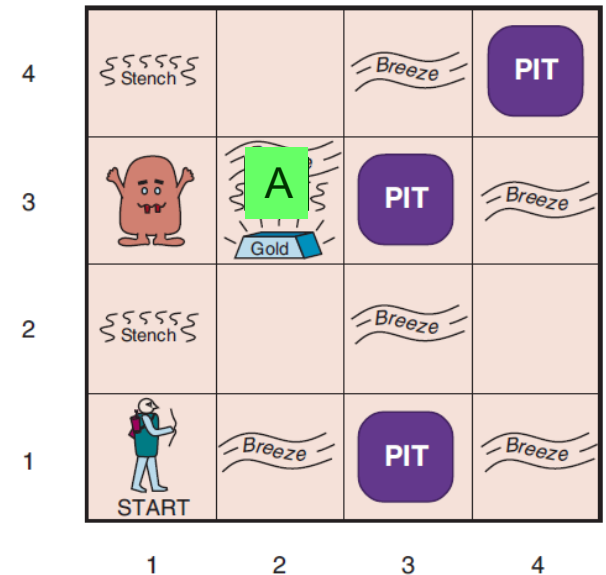
Using First-Order Logic (FOL)

Outline

- I. The wumpus world in FOL
- II. Knowledge engineering process
- III. Instantiation and Skolemization

I. The Wumpus World in FOL

First-order logic axioms are much more concise than propositional axioms.

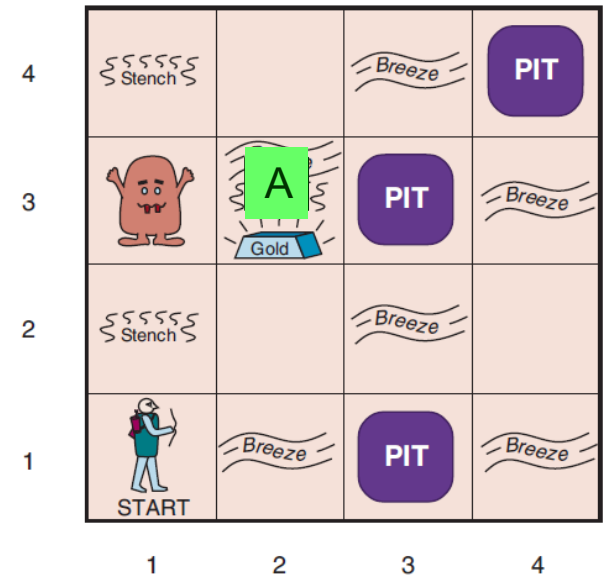


I. The Wumpus World in FOL

First-order logic axioms are much more concise than propositional axioms.

- Predicate for percept

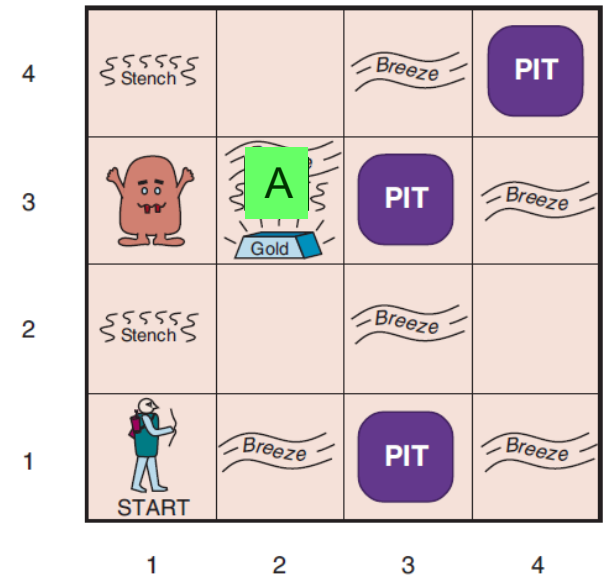
Percept([Stench, Breeze, Glitter, None, None], 5)



I. The Wumpus World in FOL

First-order logic axioms are much more concise than propositional axioms.

- Predicate for percept
- No Bump* *No Scream*
- Percept([Stench, Breeze, Glitter, None, None], 5)*
- sensor reading



I. The Wumpus World in FOL

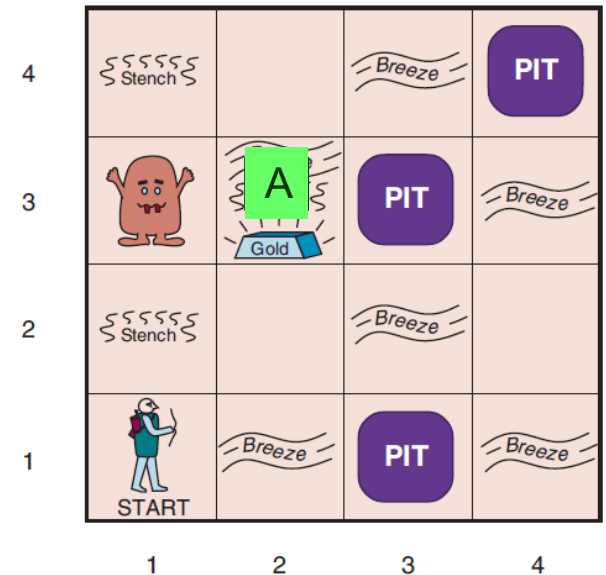
First-order logic axioms are much more concise than propositional axioms.

- Predicate for percept

Percept([*Stench*, *Breeze*, *Glitter*, *None*, *None*], 5)

sensor reading

time



I. The Wumpus World in FOL

First-order logic axioms are much more concise than propositional axioms.

- Predicate for percept

Percept([*Stench*, *Breeze*, *Glitter*, *None*, *None*], 5)

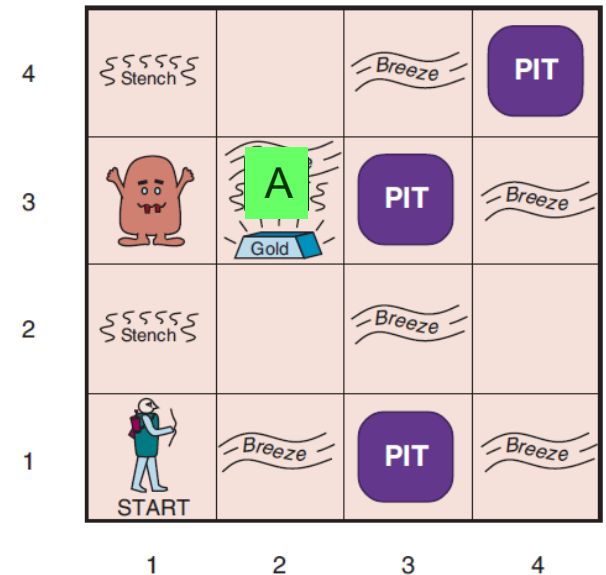
No Bump *No Scream*

- Actions

sensor reading

time

Turn(*Right*), *Turn*(*Left*), *Forward*, *Shoot*, *Grab*, *Climb*



I. The Wumpus World in FOL

First-order logic axioms are much more concise than propositional axioms.

- Predicate for percept

Percept([*Stench*, *Breeze*, *Glitter*, *None*, *None*], 5)

No Bump *No Scream*

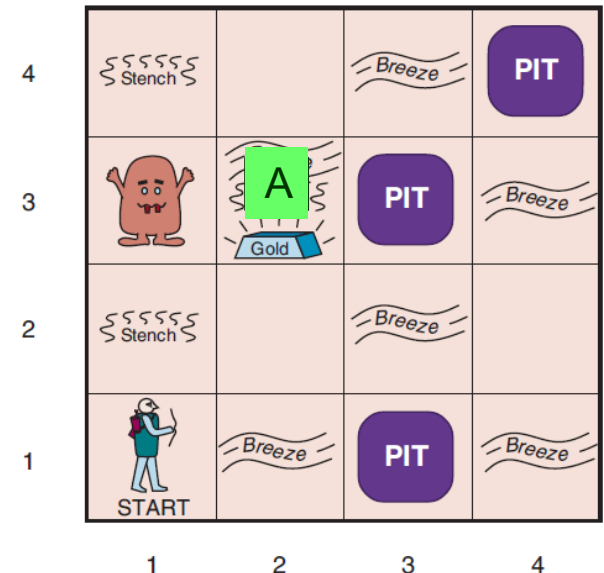
- Actions

Turn(*Right*), *Turn*(*Left*), *Forward*, *Shoot*, *Grab*, *Climb*

- Querying the KB for best action

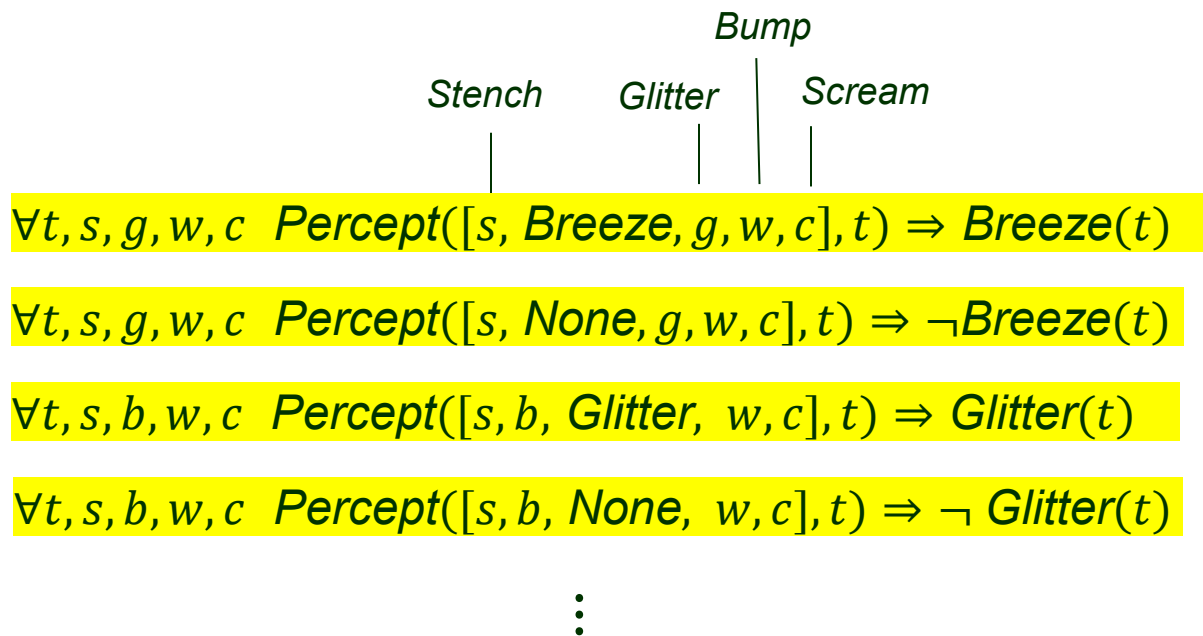
ASKVARS(*KB*, *BestAction*(*a*, 5))

A binding list, e.g., {*a/Grab*}, is returned.



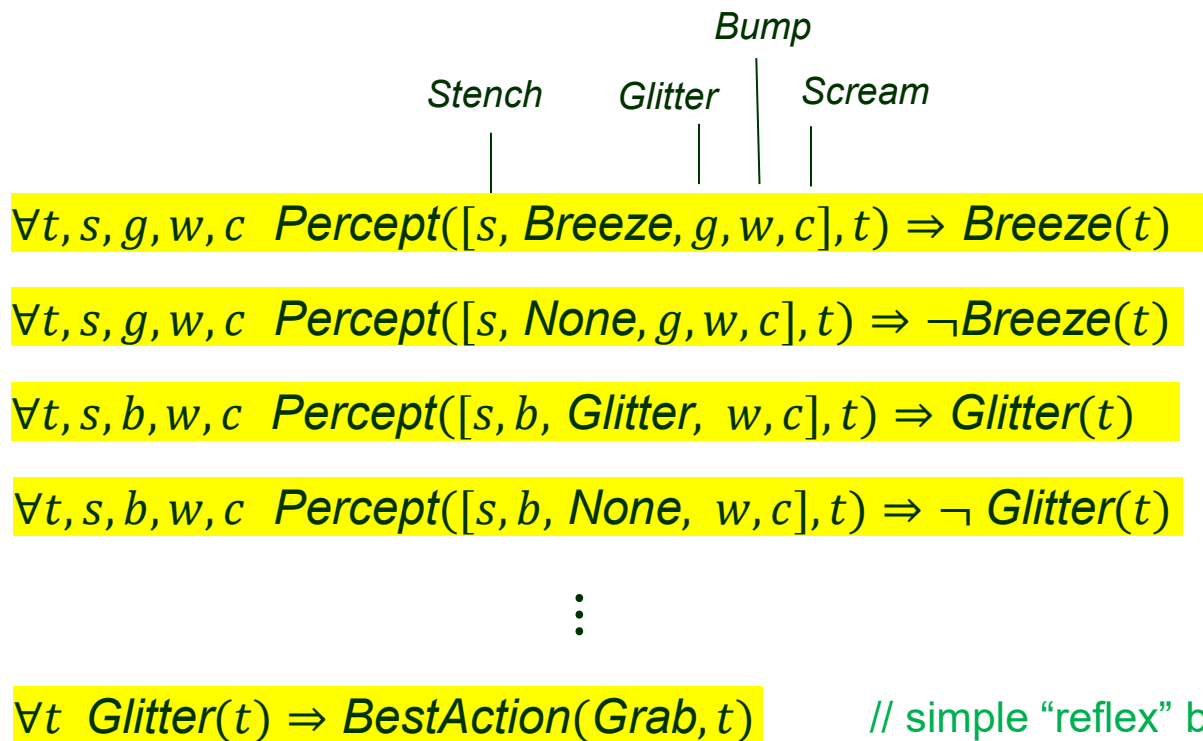
Input/Output Rules

No need for the use of fluents (e.g., $Forward^t$, $Bump^{t+1}$)!



Input/Output Rules

No need for the use of fluents (e.g., $Forward^t$, $Bump^{t+1}$)!



Rules for the Environment

- Adjacency of two squares

square at row x and column y

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

// if using propositional logic, we would have to name every square, say, $\text{Square}_{i,j}$,
// for $1 \leq i, j \leq 4$, and would need one such fact for 120 different pairs of squares!

Rules for the Environment

- Adjacency of two squares

square at row x and column y

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

// if using propositional logic, we would have to name every square, say, $\text{Square}_{i,j}$,
// for $1 \leq i, j \leq 4$, and would need one such fact for 120 different pairs of squares!

- Unary predicate *Pit* (no reason to distinguish among pits).

Rules for the Environment

- Adjacency of two squares

square at row x and column y

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

// if using propositional logic, we would have to name every square, say, $\text{Square}_{i,j}$,
// for $1 \leq i, j \leq 4$, and would need one such fact for 120 different pairs of squares!

- Unary predicate *Pit* (no reason to distinguish among pits).

$Pit([x, y]) \equiv \text{true}$ if and only if the square $[x, y]$ contains a pit.

Rules for the Environment

- Adjacency of two squares

square at row x and column y

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

// if using propositional logic, we would have to name every square, say, $\text{Square}_{i,j}$,
// for $1 \leq i, j \leq 4$, and would need one such fact for 120 different pairs of squares!

- Unary predicate *Pit* (no reason to distinguish among pits).

$Pit([x, y]) \equiv \text{true}$ if and only if the square $[x, y]$ contains a pit.

- Constants *Agent*, *Wumpus*

Rules for the Environment

- Adjacency of two squares

square at row x and column y

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

// if using propositional logic, we would have to name every square, say, $\text{Square}_{i,j}$,
// for $1 \leq i, j \leq 4$, and would need one such fact for 120 different pairs of squares!

- Unary predicate Pit (no reason to distinguish among pits).

$Pit([x, y]) \equiv \text{true}$ if and only if the square $[x, y]$ contains a pit.

- Constants $Agent$, $Wumpus$
- Ternary predicate At to represent changing or non-changing locations

$\forall t \text{ At}(Wumpus, [1, 3], t)$ // fixed location for the wumpus

Rules for the Environment

- Adjacency of two squares

square at row x and column y

$$\forall x, y, a, b \text{ Adjacent}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$$

// if using propositional logic, we would have to name every square, say, $\text{Square}_{i,j}$,
// for $1 \leq i, j \leq 4$, and would need one such fact for 120 different pairs of squares!

- Unary predicate Pit (no reason to distinguish among pits).

$Pit([x, y]) \equiv \text{true}$ if and only if the square $[x, y]$ contains a pit.

- Constants $Agent$, $Wumpus$
- Ternary predicate At to represent changing or non-changing locations

$\forall t \text{ At}(Wumpus, [1, 3], t)$ // fixed location for the wumpus

$\forall x, s_1, s_2, t \text{ At}(x, s_1, t) \wedge \text{At}(x, s_2, t) \Rightarrow s_1 = s_2$ // only one location at a time

Rules for Percepts, Actions and Inferences

- Percepts of breeze, stench, etc.

$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$$

Rules for Percepts, Actions and Inferences

- Percepts of breeze, stench, etc.

$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$$

- *Diagnostic* (inferring cause from effect)

$$\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r, s) \wedge \text{Pit}(r)$$

// if using propositional logic, we would need a separate axiom for every square.

Rules for Percepts, Actions and Inferences

- Percepts of breeze, stench, etc.

$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$$

- *Diagnostic* (inferring cause from effect)

$$\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r, s) \wedge \text{Pit}(r)$$

// if using propositional logic, we would need a separate axiom for every square.

- *Causal* (inferring effect from cause)

$$\forall s (\forall r \text{ Adjacent}(r, s) \Rightarrow \neg \text{Pit}(r)) \Rightarrow \neg \text{Breezy}(s)$$

Rules for Percepts, Actions and Inferences

- Percepts of breeze, stench, etc.

$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$$

- *Diagnostic* (inferring cause from effect)

$$\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r, s) \wedge \text{Pit}(r)$$

// if using propositional logic, we would need a separate axiom for every square.

- *Causal* (inferring effect from cause)

$$\forall s (\forall r \text{ Adjacent}(r, s) \Rightarrow \neg \text{Pit}(r)) \Rightarrow \neg \text{Breezy}(s)$$

- Quantification over time

$$\forall t \text{ HaveArrow}(t + 1) \Leftrightarrow \text{HaveArrow}(t) \wedge \neg \text{Action}(\text{Shoot}, t)$$

Rules for Percepts, Actions and Inferences

- Percepts of breeze, stench, etc.

$$\forall s, t \text{ At}(\text{Agent}, s, t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$$

- *Diagnostic* (inferring cause from effect)

$$\forall s \text{ Breezy}(s) \Leftrightarrow \exists r \text{ Adjacent}(r, s) \wedge \text{Pit}(r)$$

// if using propositional logic, we would need a separate axiom for every square.

- *Causal* (inferring effect from cause)

$$\forall s (\forall r \text{ Adjacent}(r, s) \Rightarrow \neg \text{Pit}(r)) \Rightarrow \neg \text{Breezy}(s)$$

- Quantification over time

$$\forall t \text{ HaveArrow}(t + 1) \Leftrightarrow \text{HaveArrow}(t) \wedge \neg \text{Action}(\text{Shoot}, t)$$

The first-order logic formulation is no less concise than the English description.

II. Knowledge Engineering Process

♣ Identification of questions and facts.

What will the KB support and what facts are available?

II. Knowledge Engineering Process

♣ Identification of questions and facts.

What will the KB support and what facts are available?

♣ Knowledge assembly or acquisition.

Work with real experts to extract knowledge (not yet formally represented).

II. Knowledge Engineering Process

♣ Identification of questions and facts.

What will the KB support and what facts are available?

♣ Knowledge assembly or acquisition.

Work with real experts to extract knowledge (not yet formally represented).

♣ Decision on a vocabulary (predicates, functions, and constants).

In the wumpus world, should pits be represented by objects or by a unary predicate on squares?

II. Knowledge Engineering Process

- ♣ Identification of questions and facts.

What will the KB support and what facts are available?

- ♣ Knowledge assembly or acquisition.

Work with real experts to extract knowledge (not yet formally represented).

- ♣ Decision on a vocabulary (predicates, functions, and constants).

In the wumpus world, should pits be represented by objects or by a unary predicate on squares?

- ♣ Encoding of general knowledge (axioms).

- ♣ Formal description of the problem instance.

II. Knowledge Engineering Process

♣ Identification of questions and facts.

What will the KB support and what facts are available?

♣ Knowledge assembly or acquisition.

Work with real experts to extract knowledge (not yet formally represented).

♣ Decision on a vocabulary (predicates, functions, and constants).

In the wumpus world, should pits be represented by objects or by a unary predicate on squares?

♣ Encoding of general knowledge (axioms).

♣ Formal description of the problem instance.

♣ Queries to and answers from the inference procedure.

Derive the facts we are interested in knowing.

II. Knowledge Engineering Process

♣ Identification of questions and facts.

What will the KB support and what facts are available?

♣ Knowledge assembly or acquisition.

Work with real experts to extract knowledge (not yet formally represented).

♣ Decision on a vocabulary (predicates, functions, and constants).

In the wumpus world, should pits be represented by objects or by a unary predicate on squares?

♣ Encoding of general knowledge (axioms).

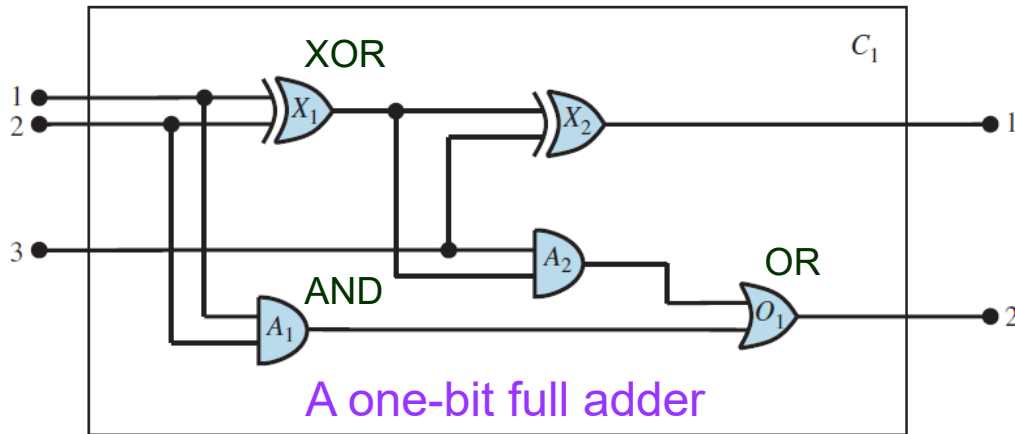
♣ Formal description of the problem instance.

♣ Queries to and answers from the inference procedure.

Derive the facts we are interested in knowing.

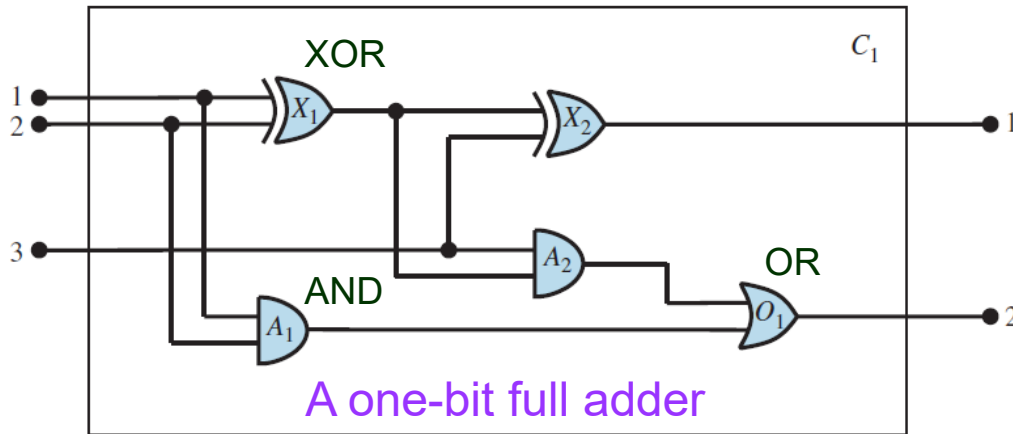
♣ Debugging and evaluation of the KB.

The Electronic Circuits Domain



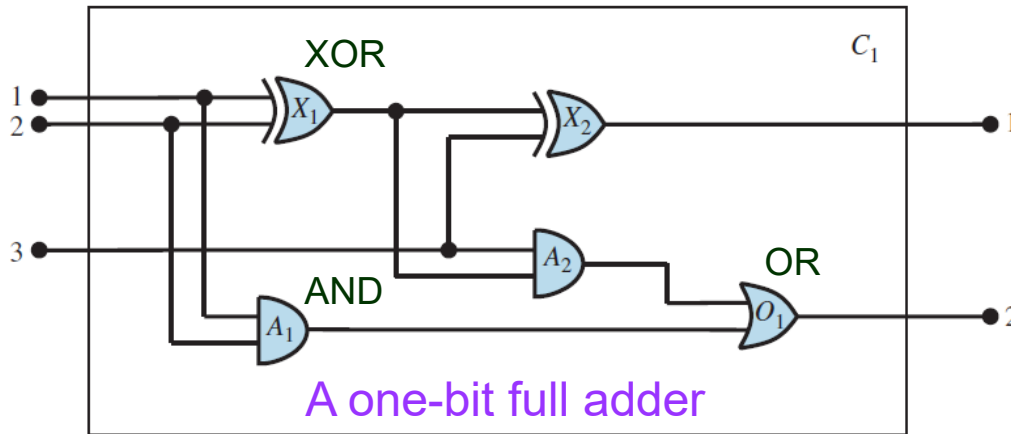
- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.

The Electronic Circuits Domain



- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.
- No need to represent wires or their paths.
- Component sizes, shapes, colors, or costs are irrelevant.

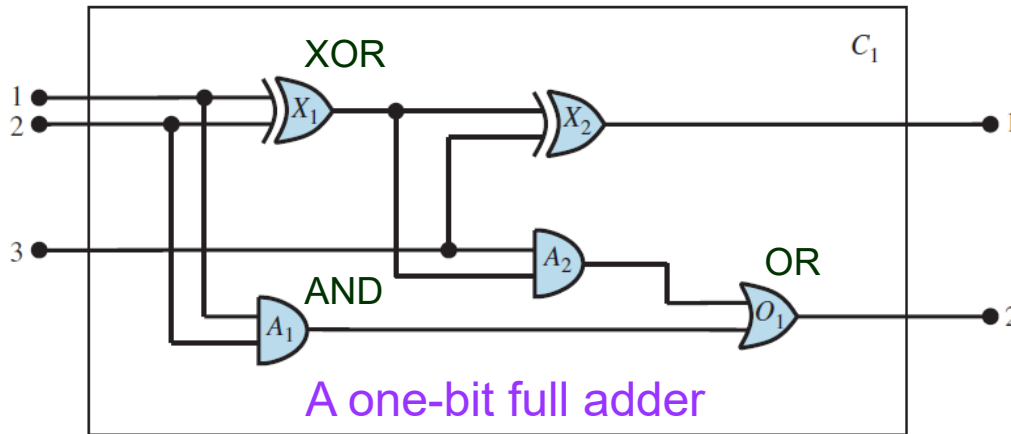
The Electronic Circuits Domain



- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.
- No need to represent wires or their paths.
- Component sizes, shapes, colors, or costs are irrelevant.

◆ Objects: actual gates and circuits. E.g., X_1 , A_2 , C_1 .

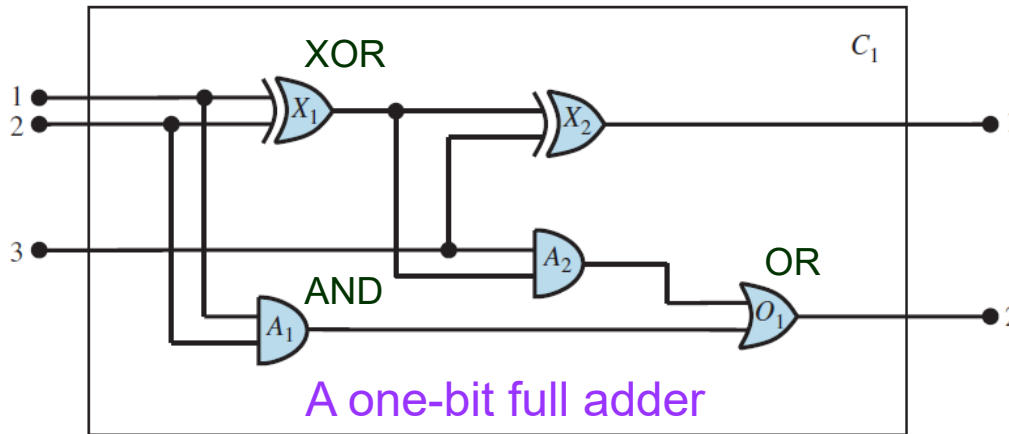
The Electronic Circuits Domain



- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.
- No need to represent wires or their paths.
- Component sizes, shapes, colors, or costs are irrelevant.

- ◆ Objects: actual gates and circuits. E.g., X_1 , A_2 , C_1 .
- ◆ Predicates: *Gate*, *Terminal*, *Circuit*

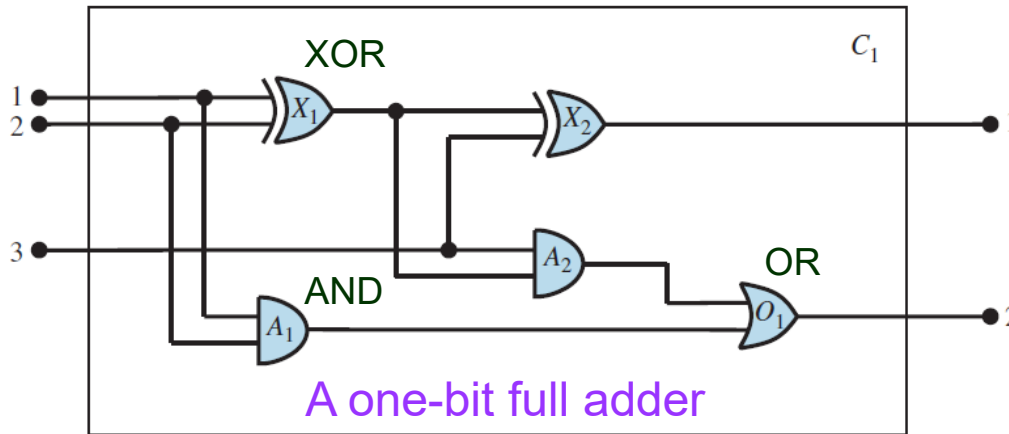
The Electronic Circuits Domain



- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.
- No need to represent wires or their paths.
- Component sizes, shapes, colors, or costs are irrelevant.

- ◆ Objects: actual gates and circuits. E.g., X_1 , A_2 , C_1 .
- ◆ Predicates: *Gate*, *Terminal*, *Circuit*
- ◆ Functions
 - ♣ *Type*(X_1): type of gate X_1 (which is *XOR*)
 - ♣ *In*(1, X_2): 1st input terminal for gate X_2
 - ♣ *Out*(2, C_1): 2nd output terminal for circuit C_1
 - ♣ *Arity*(A_1 , 2, 1): two input terminals and one output terminal for the gate A_1

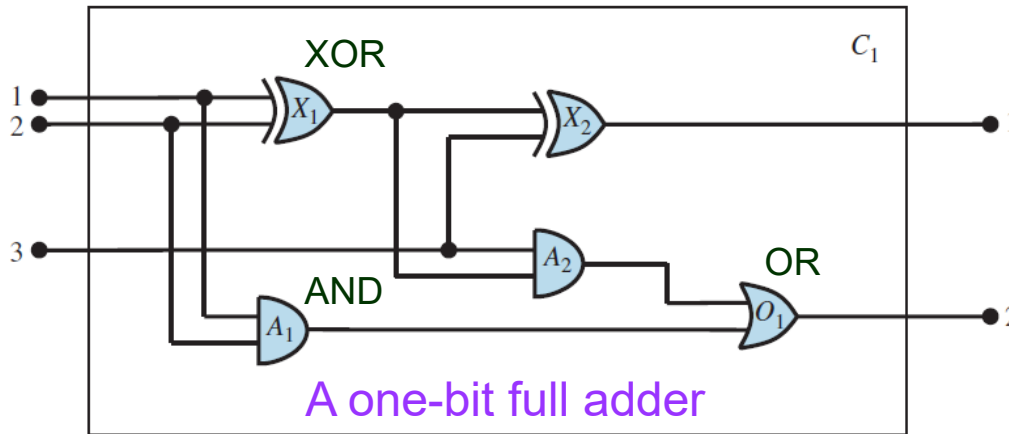
The Electronic Circuits Domain



- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.
- No need to represent wires or their paths.
- Component sizes, shapes, colors, or costs are irrelevant.

- ◆ Objects: actual gates and circuits. E.g., X_1 , A_2 , C_1 .
- ◆ Predicates: *Gate*, *Terminal*, *Circuit*
- ◆ Functions
 - ♣ *Type*(X_1): type of gate X_1 (which is *XOR*)
 - ♣ *In*(1, X_2): 1st input terminal for gate X_2
 - ♣ *Out*(2, C_1): 2nd output terminal for circuit C_1
 - ♣ *Arity*(A_1 , 2, 1): two input terminals and one output terminal for the gate A_1
- ◆ Connectivity predicate: *Connected*(*Out*(1, X_1), *In*(1, X_2))

The Electronic Circuits Domain



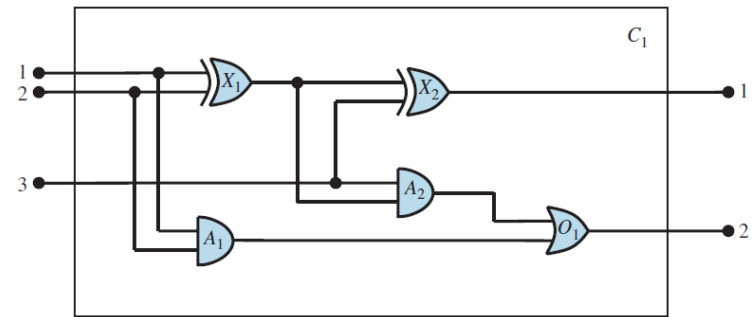
- Four types of gates: *AND*, *OR*, *XOR*, *NOT*.
1 or 2 inputs, and 1 output
- Represent connections between terminals.
- No need to represent wires or their paths.
- Component sizes, shapes, colors, or costs are irrelevant.

- ◆ Objects: actual gates and circuits. E.g., X_1 , A_2 , C_1 .
- ◆ Predicates: *Gate*, *Terminal*, *Circuit*
- ◆ Functions
 - ♣ *Type*(X_1): type of gate X_1 (which is *XOR*)
 - ♣ *In*(1, X_2): 1st input terminal for gate X_2
 - ♣ *Out*(2, C_1): 2nd output terminal for circuit C_1
 - ♣ *Arity*(A_1 , 2, 1): two input terminals and one output terminal for the gate A_1
- ◆ Connectivity predicate: *Connected*(*Out*(1, X_1), *In*(1, X_2))
- ◆ Signal function: *Signal*(t) has value 1 or 0 at time t .

Encoding General Domain Knowledge

// Two connected terminals have the same signal.

$\forall t_1, t_2 \quad \text{Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$



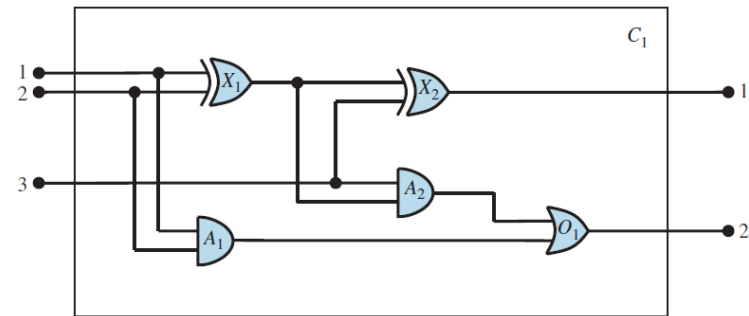
Encoding General Domain Knowledge

// Two connected terminals have the same signal.

$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$

// Every terminal has signal that is either 1 or 0.

$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0$



Encoding General Domain Knowledge

// Two connected terminals have the same signal.

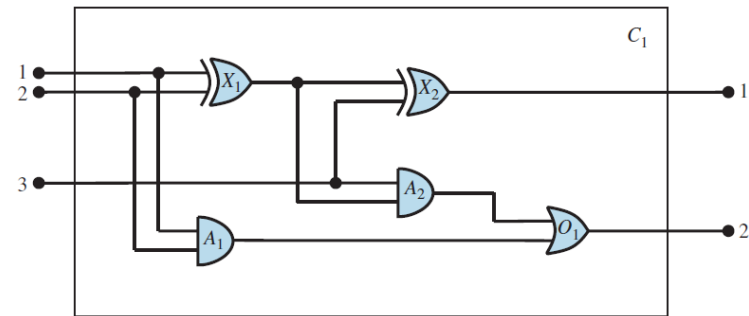
$\forall t_1, t_2 \quad \text{Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$

// Every terminal has signal that is either 1 or 0.

$\forall t \quad \text{Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0$

// Connectivity is commutative.

$\forall t_1, t_2 \quad \text{Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1)$



Encoding General Domain Knowledge

// Two connected terminals have the same signal.

$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$

// Every terminal has signal that is either 1 or 0.

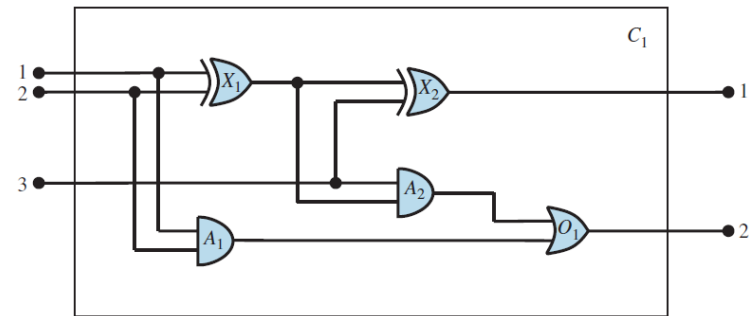
$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0$

// Connectivity is commutative.

$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1)$

// Four types of gates.

$\forall g, k \text{ Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT}$



Encoding General Domain Knowledge

// Two connected terminals have the same signal.

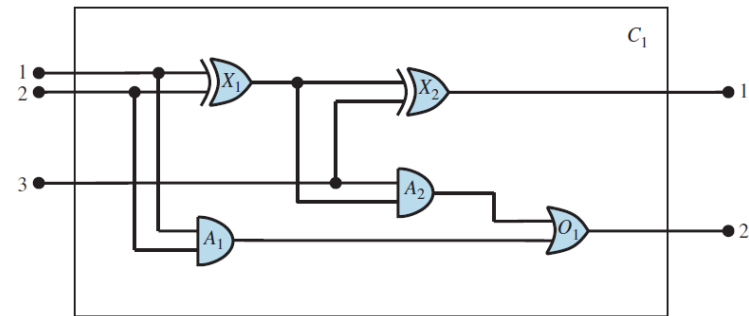
$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$

// Every terminal has signal that is either 1 or 0.

$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0$

// Connectivity is commutative.

$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1)$



// Four types of gates.

$\forall g, k \text{ Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT}$

// An AND gate outputs 0 if and only if any of its input is 0.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{AND} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0)$

Encoding General Domain Knowledge

// Two connected terminals have the same signal.

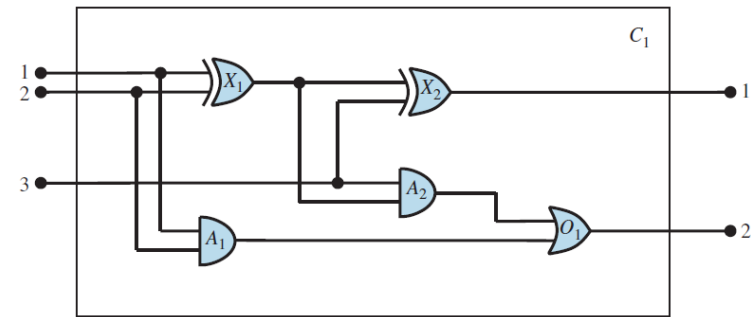
$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$

// Every terminal has signal that is either 1 or 0.

$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0$

// Connectivity is commutative.

$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1)$



// Four types of gates.

$\forall g, k \text{ Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT}$

// An AND gate outputs 0 if and only if any of its input is 0.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{AND} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0)$

// An OR gate outputs 1 if and only if any of its input is 1.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{OR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 1)$

Encoding General Domain Knowledge

// Two connected terminals have the same signal.

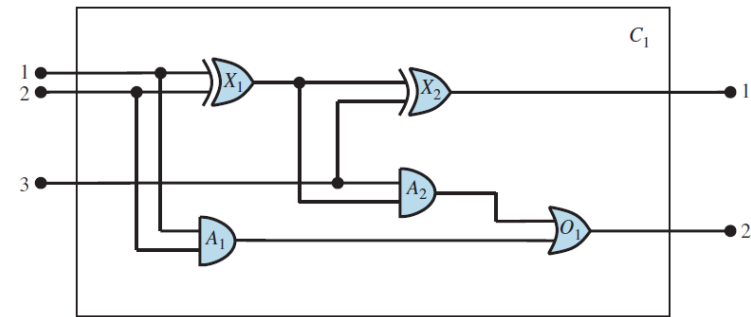
$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2)$

// Every terminal has signal that is either 1 or 0.

$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0$

// Connectivity is commutative.

$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1)$



// Four types of gates.

$\forall g, k \text{ Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT}$ (errors or typos in the textbook)

// An AND gate outputs 0 if and only if any of its input is 0.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{AND} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0)$

// An OR gate outputs 1 if and only if any of its input is 1.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{OR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 1)$

cont'd

// An XOR gate outputs 1 if and only if its inputs are different.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)))$

// An NOT gate's output is different from its input.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{Out}(1, g)) \neq \text{Signal}(\text{In}(1, g)))$

cont'd

// An XOR gate outputs 1 if and only if its inputs are different.

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)))$$

// An NOT gate's output is different from its input.

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(1, g)))$$

// All the gates (except for NOT) have two inputs and one output.

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) \neq \text{NOT} \Rightarrow \text{Arity}(g, 1, 1)$$

$$\forall g, k \text{ Gate}(g) \wedge k \neq \text{Type}(g) \wedge (k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR}) \Rightarrow \text{Arity}(g, 2, 1)$$

cont'd

// An XOR gate outputs 1 if and only if its inputs are different.

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)))$$

// An NOT gate's output is different from its input.

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(1, g)))$$

// All the gates (except for NOT) have two inputs and one output.

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow \text{Arity}(g, 1, 1)$$

$$\forall g, k \text{ Gate}(g) \wedge k \neq \text{Type}(g) \wedge (k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR}) \Rightarrow \text{Arity}(g, 2, 1)$$

// A circuit has terminals exactly up to its input and output arity.

$$\begin{aligned} & \forall c, i, j \text{ Circuit}(c) \wedge \text{Arity}(c, i, j) \Rightarrow \\ & \forall n (n \leq i \Rightarrow \text{Terminal}(\text{In}(n, c)) \wedge (n > i \Rightarrow \text{In}(n, c) = \text{Nothing})) \wedge \\ & \forall n (n \leq j \Rightarrow \text{Terminal}(\text{Out}(n, c)) \wedge (n > j \Rightarrow \text{Out}(n, c) = \text{Nothing})) \end{aligned}$$

cont'd

// An XOR gate outputs 1 if and only if its inputs are different.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)))$

// An NOT gate's output is different from its input.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(1, g)))$

// All the gates (except for NOT) have two inputs and one output.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow \text{Arity}(g, 1, 1)$

$\forall g, k \text{ Gate}(g) \wedge k = \text{Type}(g) \wedge (k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR}) \Rightarrow \text{Arity}(g, 2, 1)$

// A circuit has terminals exactly up to its input and output arity.

$\forall c, i, j \text{ Circuit}(c) \wedge \text{Arity}(c, i, j) \Rightarrow$
 $\forall n (n \leq i \Rightarrow \text{Terminal}(\text{In}(n, c)) \wedge (n > i \Rightarrow \text{In}(n, c) = \text{Nothing})) \wedge$
 $\forall n (n \leq j \Rightarrow \text{Terminal}(\text{Out}(n, c)) \wedge (n > j \Rightarrow \text{Out}(n, c) = \text{Nothing}))$

// Gates and terminals are all distinct.

$\forall g, t, s \text{ Gate}(g) \wedge \text{Terminal}(t) \wedge \text{Signal}(s) \Rightarrow g \neq t \wedge g \neq s \wedge t \neq s$

function not predicate

cont'd

// An XOR gate outputs 1 if and only if its inputs are different.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)))$

// An NOT gate's output is different from its input.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow (\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(1, g)))$

// All the gates (except for NOT) have two inputs and one output.

$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow \text{Arity}(g, 1, 1)$

$\forall g, k \text{ Gate}(g) \wedge k = \text{Type}(g) \wedge (k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR}) \Rightarrow \text{Arity}(g, 2, 1)$

// A circuit has terminals exactly up to its input and output arity.

$\forall c, i, j \text{ Circuit}(c) \wedge \text{Arity}(c, i, j) \Rightarrow$
 $\forall n (n \leq i \Rightarrow \text{Terminal}(\text{In}(n, c)) \wedge (n > i \Rightarrow \text{In}(n, c) = \text{Nothing})) \wedge$
 $\forall n (n \leq j \Rightarrow \text{Terminal}(\text{Out}(n, c)) \wedge (n > j \Rightarrow \text{Out}(n, c) = \text{Nothing}))$

// Gates and terminals are all distinct.

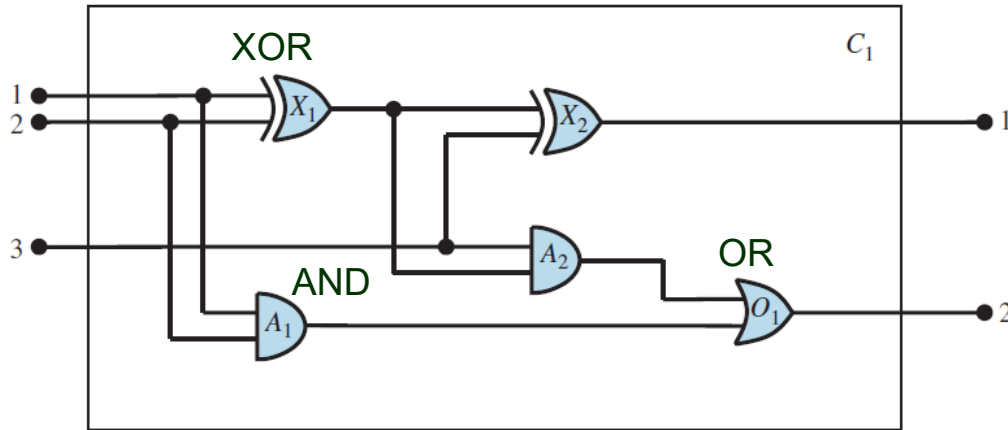
$\forall g, t, s \text{ Gate}(g) \wedge \text{Terminal}(t) \wedge \text{Signal}(s) \Rightarrow g \neq t \wedge g \neq s \wedge t \neq s$

// Gates are circuits.

$\forall g \text{ Gate}(g) \Rightarrow \text{Circuit}(g)$

function not predicate

Encoding a Problem Instance



◆ Circuit and component gates:

$Circuit(C_1) \wedge Arity(C_1, 3, 2)$

$Gate(X_1) \wedge Type(X_1) = XOR$

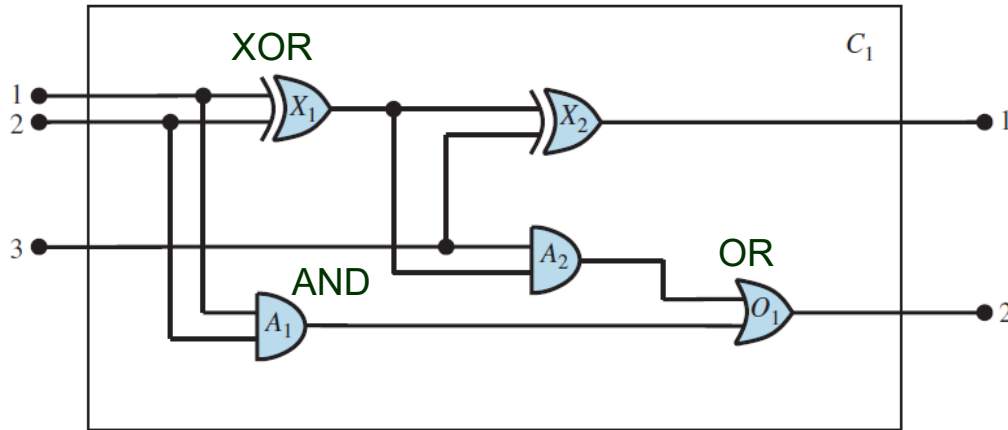
$Gate(X_2) \wedge Type(X_2) = XOR$

$Gate(A_1) \wedge Type(A_1) = AND$

$Gate(A_2) \wedge Type(A_2) = AND$

$Gate(O_1) \wedge Type(O_1) = OR$

Encoding a Problem Instance



◆ Circuit and component gates:

$Circuit(C_1) \wedge Arity(C_1, 3, 2)$

$Gate(X_1) \wedge Type(X_1) = XOR$

$Gate(X_2) \wedge Type(X_2) = XOR$

$Gate(A_1) \wedge Type(A_1) = AND$

$Gate(A_2) \wedge Type(A_2) = AND$

$Gate(O_1) \wedge Type(O_1) = OR$

◆ Connections between the circuit and component gates:

$Connected(Out(1, X_1), In(1, X_2))$

$Connected(Out(1, X_1), In(2, A_2))$

$Connected(Out(1, A_2), In(1, O_1))$

$Connected(Out(1, A_1), In(2, O_1))$

$Connected(Out(1, X_2), Out(1, C_1))$

$Connected(Out(1, O_1), Out(2, C_1))$

$Connected(In(1, C_1), In(1, X_1))$

$Connected(In(1, C_1), In(1, A_1))$

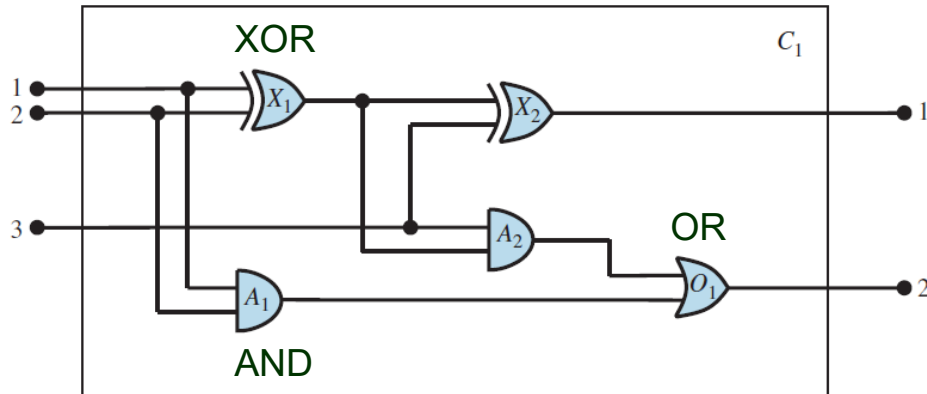
$Connected(In(2, C_1), In(2, X_1))$

$Connected(In(2, C_1), In(2, A_1))$

$Connected(In(3, C_1), In(2, X_2))$

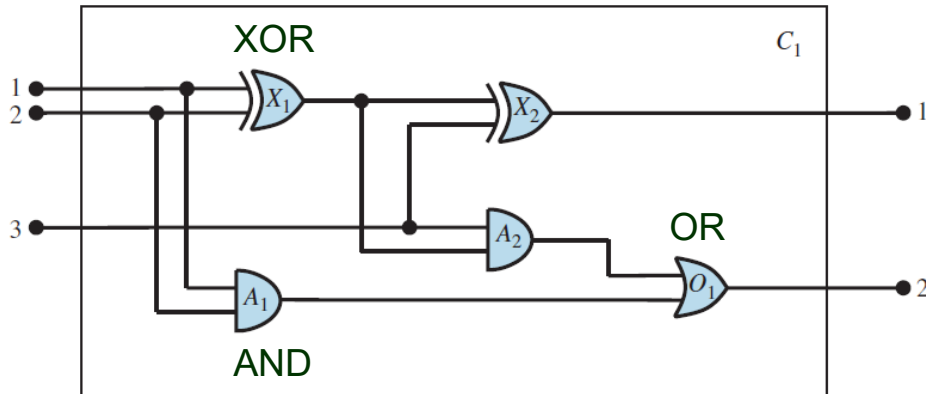
$Connected(In(3, C_1), In(1, A_2))$

Queries



Q. What combinations of inputs would cause the first output of C_1 to be 0 and its second output to be 1?

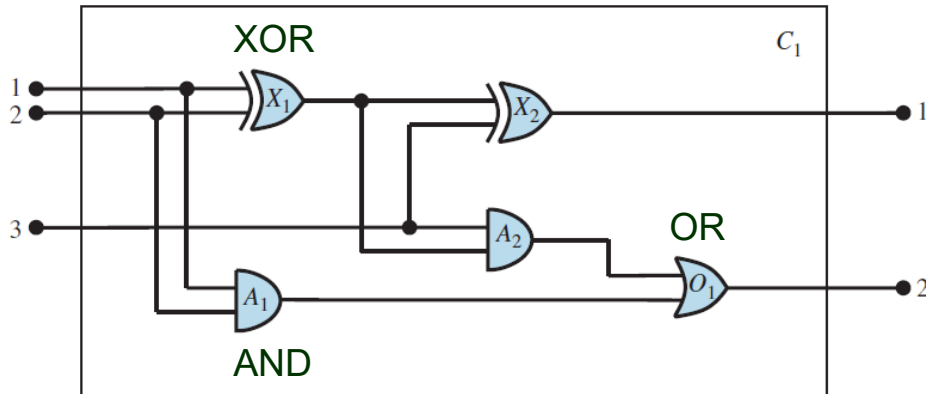
Queries



Q. What combinations of inputs would cause the first output of C_1 to be 0 and its second output to be 1?

$$\exists i_1, i_2, i_3 \text{ } \text{Signal}(\text{In}(1, C_1)) = i_1 \wedge \text{Signal}(\text{In}(2, C_1)) = i_2 \wedge \text{Signal}(\text{In}(3, C_1)) = i_3 \\ \wedge \text{Signal}(\text{Out}(1, C_1)) = 0 \wedge \text{Signal}(\text{Out}(2, C_1)) = 1$$

Queries



Q. What combinations of inputs would cause the first output of C_1 to be 0 and its second output to be 1?

$$\exists i_1, i_2, i_3 \text{ } \text{Signal}(\text{In}(1, C_1)) = i_1 \wedge \text{Signal}(\text{In}(2, C_1)) = i_2 \wedge \text{Signal}(\text{In}(3, C_1)) = i_3 \\ \wedge \text{Signal}(\text{Out}(1, C_1)) = 0 \wedge \text{Signal}(\text{Out}(2, C_1)) = 1$$

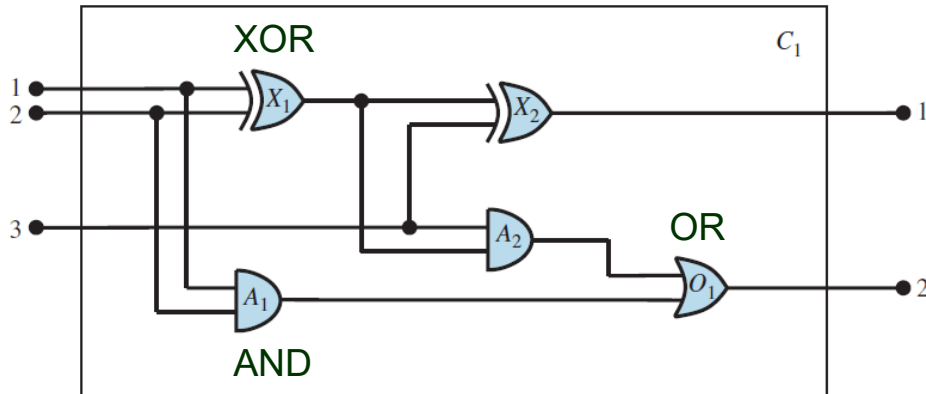
ASKVARS will give three substitutions as answers.

$$\{i_1/1, i_2/1, i_3/0\}$$

$$\{i_1/1, i_2/0, i_3/1\}$$

$$\{i_1/0, i_2/1, i_3/1\}$$

Queries



Q. What combinations of inputs would cause the first output of C_1 to be 0 and its second output to be 1?

$$\exists i_1, i_2, i_3 \text{ Signal(In}(1, C_1)) = i_1 \wedge \text{Signal(In}(2, C_1)) = i_2 \wedge \text{Signal(In}(3, C_1)) = i_3 \\ \wedge \text{Signal(Out}(1, C_1)) = 0 \wedge \text{Signal(Out}(2, C_1)) = 1$$

ASKVARS will give three substitutions as answers.

$$\{i_1/1, i_2/1, i_3/0\}$$

$$\{i_1/1, i_2/0, i_3/1\}$$

$$\{i_1/0, i_2/1, i_3/1\}$$

Debugging: We can also perturb the KB to see what erroneous behaviors would emerge, and then identify missing rules for instance.

III. Reduction to Propositional Inference

One way of inference is to convert the first-order KB to propositional logic.

III. Reduction to Propositional Inference

One way of inference is to convert the first-order KB to propositional logic.

- Eliminate universal quantifiers.

$\forall x \text{ Human}(x) \Rightarrow \text{Fallible}(x)$

// All humans are fallible.

III. Reduction to Propositional Inference

One way of inference is to convert the first-order KB to propositional logic.

- Eliminate universal quantifiers.

$\forall x \text{ Human}(x) \Rightarrow \text{Fallible}(x)$ // All humans are fallible.

We can infer sentences like

$\text{Human}(\text{Socrates}) \Rightarrow \text{Fallible}(\text{Socrates})$

$\text{Human}(\text{Einstein}) \Rightarrow \text{Fallible}(\text{Einstein})$

$\text{Human}(\text{Messi}) \Rightarrow \text{Fallible}(\text{Messi})$

$\vdots$

III. Reduction to Propositional Inference

One way of inference is to convert the first-order KB to propositional logic.

- Eliminate universal quantifiers.

$\forall x \text{ Human}(x) \Rightarrow \text{Fallible}(x)$ // All humans are fallible.

We can infer sentences like

$\text{Human}(\text{Socrates}) \Rightarrow \text{Fallible}(\text{Socrates})$

$\text{Human}(\text{Einstein}) \Rightarrow \text{Fallible}(\text{Einstein})$

$\text{Human}(\text{Messi}) \Rightarrow \text{Fallible}(\text{Messi})$

$\vdots$

From

// All birds are warm-blooded and have wings.

$\forall x \text{ Bird}(x) \Rightarrow \text{WarmBlooded}(x) \wedge \text{HaveWings}(x)$

III. Reduction to Propositional Inference

One way of inference is to convert the first-order KB to propositional logic.

- Eliminate universal quantifiers.

$\forall x \text{ Human}(x) \Rightarrow \text{Fallible}(x)$ // All humans are fallible.

We can infer sentences like

$\text{Human}(\text{Socrates}) \Rightarrow \text{Fallible}(\text{Socrates})$

$\text{Human}(\text{Einstein}) \Rightarrow \text{Fallible}(\text{Einstein})$

$\text{Human}(\text{Messi}) \Rightarrow \text{Fallible}(\text{Messi})$

$\vdots$

From

// All birds are warm-blooded and have wings.

$\forall x \text{ Bird}(x) \Rightarrow \text{WarmBlooded}(x) \wedge \text{HaveWings}(x)$

We can infer (if $\text{Bird}(\text{Ostrich})$ and $\text{Bird}(\text{Peacock})$ are in the KB):

$\text{WarmBlooded}(\text{Ostrich}) \wedge \text{HaveWings}(\text{Ostrich})$

$\text{WarmBlooded}(\text{Peacock}) \wedge \text{HaveWings}(\text{Peacock})$

Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

Substitute a ground term for a universally quantified variable.

$$\frac{\forall v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

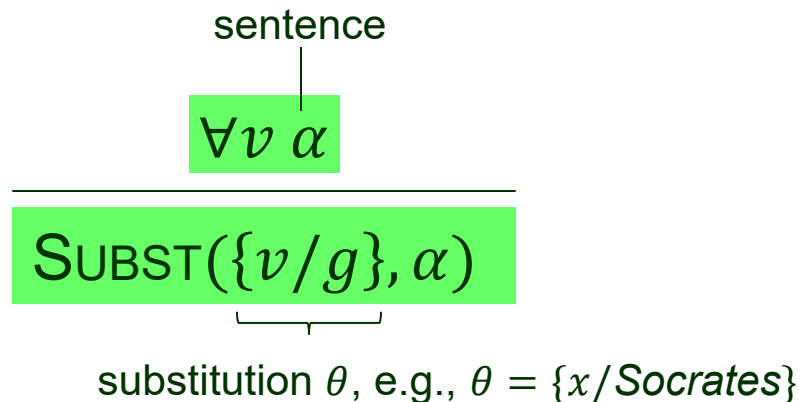
Substitute a ground term for a universally quantified variable.

$$\frac{\text{sentence} \quad \forall v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

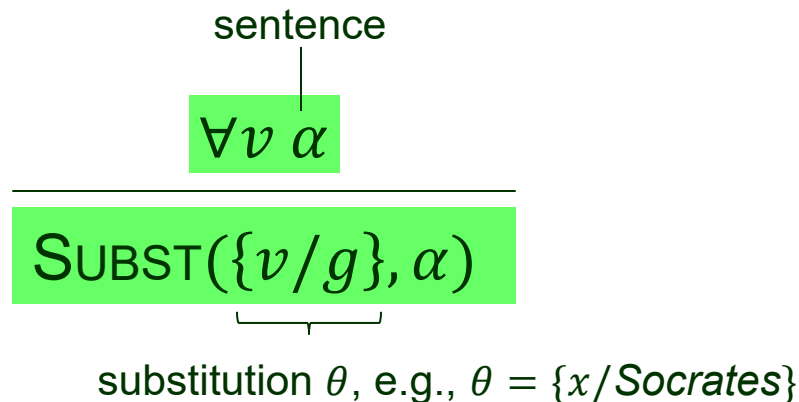
Substitute a ground term for a universally quantified variable.



Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

Substitute a ground term for a universally quantified variable.



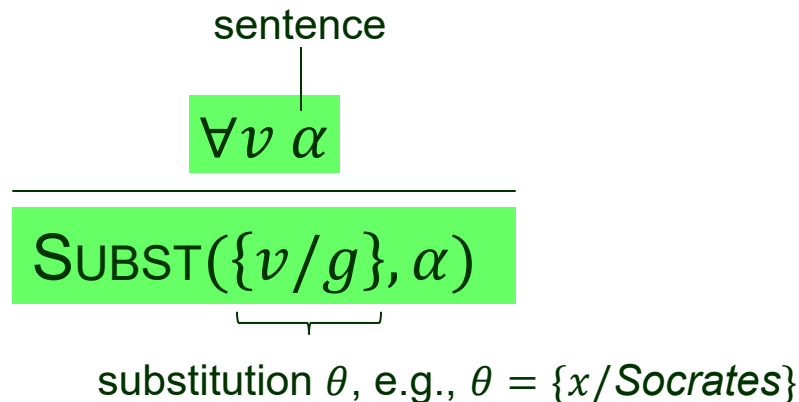
E.g., $\theta = \{x/\text{Ostrich}\}$

$\forall x \text{ Bird}(x) \Rightarrow \text{WarmBlooded}(x) \wedge \text{HaveWings}(x)$

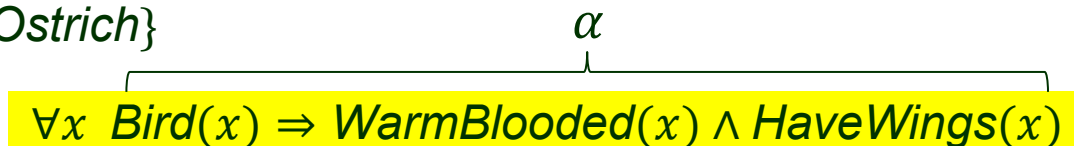
Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

Substitute a ground term for a universally quantified variable.



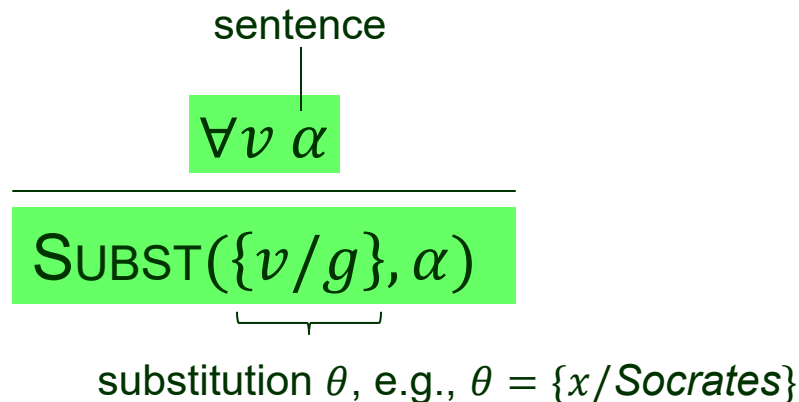
E.g., $\theta = \{x/\text{Ostrich}\}$



Universal and Existential Instantiations

A *ground term* in FOL is a term without variables.

Substitute a ground term for a universally quantified variable.



E.g., $\theta = \{x/\text{Ostrich}\}$

$$\forall x \text{ Bird}(x) \Rightarrow \text{WarmBlooded}(x) \wedge \text{HaveWings}(x)$$

$$\text{SUBST}(\theta, \alpha) \equiv \text{Bird}(\text{Ostrich}) \Rightarrow \text{WarmBlooded}(\text{Ostrich}) \wedge \text{HaveWings}(\text{Ostrich})$$

Existential Instantiation

Substitute a single **new constant symbol** for an existentially quantified variable.

$$\exists v \alpha$$

$$\text{SUBST}(\{v/g\}, \alpha)$$

Existential Instantiation

Substitute a single **new constant symbol** for an existentially quantified variable.

$$\exists v \alpha$$

$$\text{SUBST}(\{v/g\}, \alpha)$$

From

$$\exists y \text{ Mother}(y, \text{Liam})$$

Existential Instantiation

Substitute a single **new constant symbol** for an existentially quantified variable.

$$\frac{\exists v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

From

$$\exists y \text{ Mother}(y, \text{Liam})$$

we can infer

$$\text{Mother}(\text{LiamsMom}, \text{Liam})$$

as long as *LiamsMom* does **not** appear elsewhere in the KB.

Existential Instantiation

Substitute a single **new constant symbol** for an existentially quantified variable.

$$\exists v \alpha$$

$$\text{SUBST}(\{v/g\}, \alpha)$$

From

$$\exists y \text{ Mother}(y, \text{Liam})$$

we can infer

$$\text{Mother}(\text{LiamsMom}, \text{Liam})$$

as long as *LiamsMom* does **not** appear elsewhere in the KB.

|
Skolem constant

Propositionalization (Non-Standard Way)

$\forall x \forall y \text{ Ancestor}(x, y) \Rightarrow \text{Parent}(x, y) \vee \exists z (\text{Ancestor}(x, z) \wedge \text{Ancestor}(z, y))$

KB:

Ancestor(John, David)

Parent(John, David)

Parent(David, Lisa)

Propositionalization (Non-Standard Way)

$\forall x \forall y \text{ Ancestor}(x, y) \Rightarrow \text{Parent}(x, y) \vee \exists z (\text{Ancestor}(x, z) \wedge \text{Ancestor}(z, y))$

KB:

Ancestor(John, David)

Parent(John, David)

Parent(David, Lisa)

Ancestor(John, David)

$\Rightarrow \text{Parent}(\text{John}, \text{David}) \vee (\text{Ancestor}(\text{John}, \text{JohnDavidAnc}) \wedge \text{Ancestor}(\text{JohnDavidAnc}, \text{David}))$

Propositionalization (Non-Standard Way)

$\forall x \forall y \text{ Ancestor}(x, y) \Rightarrow \text{Parent}(x, y) \vee \exists z (\text{Ancestor}(x, z) \wedge \text{Ancestor}(z, y))$

KB: *Ancestor(John, David)*

Parent(John, David)

Parent(David, Lisa)

Ancestor(John, David)

$\Rightarrow \text{Parent}(\text{John}, \text{David}) \vee (\text{Ancestor}(\text{John}, \text{JohnDavidAnc}) \wedge \text{Ancestor}(\text{JohnDavidAnc}, \text{David}))$

Ancestor(David, John)

$\Rightarrow \text{Parent}(\text{David}, \text{John}) \vee (\text{Ancestor}(\text{David}, \text{DavidJohnAnc}) \wedge \text{Ancestor}(\text{DavidJohnAnc}, \text{John}))$

Ancestor(John, Lisa)

$\Rightarrow \text{Parent}(\text{John}, \text{Lisa}) \vee (\text{Ancestor}(\text{John}, \text{JohnLisaAnc}) \wedge \text{Ancestor}(\text{JohnLisaAnc}, \text{Lisa}))$

$\vdots$

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\exists y \ P(y, x_1, \dots, x_n)$$

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\exists y \ P(y, x_1, \dots, x_n) \quad // \ y \text{ depends on } x_1, \dots, x_n$$

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\exists y \ P(y, x_1, \dots, x_n) \quad // \ y \text{ depends on } x_1, \dots, x_n$$

↓ eliminate y by introducing
function f

$$P(f(x_1, \dots, x_n), x_1, \dots, x_n)$$

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\begin{array}{c} \exists y \, P(y, x_1, \dots, x_n) \quad // y \text{ depends on } x_1, \dots, x_n \\ \downarrow \text{eliminate } y \text{ by introducing} \\ \text{function } f \\ P(f(x_1, \dots, x_n), x_1, \dots, x_n) \end{array}$$

- ◆ That $P(y, x_1, \dots, x_n) = \text{true}$ implicitly defines y as a function of $x_1, \dots, x_n$ (analogous to the implicit function theorem in multivariate calculus).

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\exists y \ P(y, x_1, \dots, x_n) \quad // \ y \text{ depends on } x_1, \dots, x_n$$

↓ eliminate y by introducing
function f

$$P(f(x_1, \dots, x_n), x_1, \dots, x_n)$$

- ◆ That $P(y, x_1, \dots, x_n) = \text{true}$ implicitly defines y as a function of $x_1, \dots, x_n$ (analogous to the implicit function theorem in multivariate calculus).

$$\exists y \ \text{Mother}(y, \text{Liam})$$

$$\exists y \ \text{Mother}(y, \text{Sophia})$$

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\begin{array}{c} \exists y \, P(y, x_1, \dots, x_n) \quad // y \text{ depends on } x_1, \dots, x_n \\ \downarrow \text{eliminate } y \text{ by introducing} \\ \text{function } f \\ P(f(x_1, \dots, x_n), x_1, \dots, x_n) \end{array}$$

- ♦ That $P(y, x_1, \dots, x_n) = \text{true}$ implicitly defines y as a function of $x_1, \dots, x_n$ (analogous to the implicit function theorem in multivariate calculus).

$\exists y \, \text{Mother}(y, \text{Liam})$	$\xrightarrow{\text{new unary function } \text{mom}()}$	$\text{Mother}(\text{mom}(\text{Liam}), \text{Liam})$
$\exists y \, \text{Mother}(y, \text{Sophia})$		$\text{Mother}(\text{mom}(\text{Sophia}), \text{Sophia})$

Skolemization (Standard Way)

A more standard way to eliminate an existential quantifier is to introduce a **new function symbol**, which is, however, not applicable in generating a PL sentence.

$$\begin{array}{c} \exists y \ P(y, x_1, \dots, x_n) \quad // y \text{ depends on } x_1, \dots, x_n \\ \downarrow \text{eliminate } y \text{ by introducing} \\ \text{function } f \\ P(f(x_1, \dots, x_n), x_1, \dots, x_n) \end{array}$$

- ◆ That $P(y, x_1, \dots, x_n) = \text{true}$ implicitly defines y as a function of $x_1, \dots, x_n$ (analogous to the implicit function theorem in multivariate calculus).

$\exists y \ \text{Mother}(y, \text{Liam})$	new unary function <i>mom</i> () $\longrightarrow$	$\text{Mother}(\text{mom}(\text{Liam}), \text{Liam})$
$\exists y \ \text{Mother}(y, \text{Sophia})$		$\text{Mother}(\text{mom}(\text{Sophia}), \text{Sophia})$

- ◆ **Advantage:** one function instead of two new constants to denote the moms of Liam and Sophia.